Resumen del Proyecto SCEF

ESP32 + sensores + LCD + teclado matricial + Wi-Fi + FastAPI + PostgreSQL

1. Estado actual del proyecto

• **ESP32:** funcionando y conectado por USB en COM5.

• **DHT22:** conectado y utilizado para temperatura y humedad.

• **Sensor de lluvia:** lectura analógica funcionando mediante AO.

• **LCD 16x2 I2C:** funcionando. La dirección I2C confirmada es **0x27**.

• **Wi-Fi:** el ESP32 se conecta al hotspot del iPhone.

• **Backend:** FastAPI recibe los datos mediante HTTP POST.

• **Base de datos:** PostgreSQL está recibiendo y almacenando las lecturas.

• **Docker/pgAdmin:** utilizados para ejecutar y revisar el backend/base de datos.

• **Teclado 4x4:** conectado físicamente; la integración con el menú todavía está en pruebas.

• **Menú LCD:** ya se observa en la pantalla, por ejemplo “MENU: 1 > 4 / SENSORES”.

2. Pines conectados al ESP32

Componente	Pin del componente	ESP32 / GPIO	Función
DHT22	DATA	GPIO 4 (D4)	Temperatura y humedad
DHT22	VCC	3.3V	Alimentación
DHT22	GND	GND	Tierra
Sensor lluvia	AO	GPIO 34 (D34)	Lectura analógica
LCD I2C	SDA	GPIO 21 (D21)	Datos I2C
LCD I2C	SCL	GPIO 22 (D22)	Reloj I2C
LCD I2C	VCC	5V	Alimentación
LCD I2C	GND	GND	Tierra
Teclado 4x4	Pin 1 - naranja	GPIO 13 (D13)	Fila 1
Teclado 4x4	Pin 2 - amarillo	GPIO 14 (D14)	Fila 2
Teclado 4x4	Pin 3 - verde	GPIO 27 (D27)	Fila 3
Teclado 4x4	Pin 4 - azul	GPIO 26 (D26)	Fila 4
Teclado 4x4	Pin 5 - morado	GPIO 25 (D25)	Columna 1
Teclado 4x4	Pin 6 - gris	GPIO 33 (D33)	Columna 2
Teclado 4x4	Pin 7 - blanco	GPIO 32 (D32)	Columna 3
Teclado 4x4	Pin 8 - negro	GPIO 18 (D18)	Columna 4

3. Configuración del teclado 4x4

La configuración utilizada en Arduino IDE es:

```
const byte ROW_PINS[4] = {13, 14, 27, 26};
const byte COL_PINS[4] = {25, 33, 32, 18};

const char KEYMAP[4][4] = {
```

```
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
```

Para inicializar las filas y columnas:

```
for (byte i = 0; i < 4; i++) {
  pinMode(ROW_PINS[i], OUTPUT);
  digitalWrite(ROW_PINS[i], HIGH);

  pinMode(COL_PINS[i], INPUT_PULLUP);
}
```

4. LCD e I2C

El escáner I2C confirmó el dispositivo en **0x27**. El bus se inicializa con SDA en GPIO 21 y SCL en GPIO 22:

```
Wire.begin(21, 22);
```

El LCD actualmente muestra correctamente el menú y los textos de sensores.

5. Comunicación de red

La red utilizada actualmente es el hotspot del iPhone.

ESP32: 172.20.10.2 **PC:** 172.20.10.9

API: http://172.20.10.9:8000/api/v1/readings/

FastAPI se ejecuta con:

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Se comprobó que el ESP32 puede enviar lecturas y que FastAPI responde con **201 Created**. También se verificó que las filas de PostgreSQL aumentan cuando llegan nuevas lecturas.

6. Lógica principal de Arduino

El programa actual está organizado aproximadamente en estas partes:

• Lectura del DHT22: temperatura y humedad.

• Lectura analógica del sensor de lluvia en GPIO 34.

• Clasificación de lluvia: SECO, HUMEDO o LLUVIA.

• Actualización de la pantalla LCD.

• Conexión/reconexión Wi-Fi.

• Envío de datos JSON al endpoint de FastAPI.

• Lectura del teclado matricial.

• Procesamiento de teclas para el menú.

• Actualización de sensores cada 10 segundos.

7. Código base de configuración de Arduino IDE

Estas son las definiciones principales que deben mantenerse en el proyecto:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <DHT.h>

#define DHT_PIN 4
#define DHT_TYPE DHT22
#define RAIN_PIN 34
#define LCD_ADDR 0x27

#define GROUP_ID 1
#define DEVICE_ID 1

const byte ROW_PINS[4] = {13, 14, 27, 26};
const byte COL_PINS[4] = {25, 33, 32, 18};

const char KEYMAP[4][4] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

const char* SSID = "iPhone de Edwin";
const char* PASSWORD = "TU_CLAVE_WIFI";

const char* API_URL =
  "http://172.20.10.9:8000/api/v1/readings/";

DHT dht(DHT_PIN, DHT_TYPE);
```

8. Función actual de lectura del teclado

```
char leerTecla() {

  for (byte row = 0; row < 4; row++) {

    for (byte i = 0; i < 4; i++) {
      digitalWrite(ROW_PINS[i], HIGH);
    }

    digitalWrite(ROW_PINS[row], LOW);

    for (byte col = 0; col < 4; col++) {

      if (digitalRead(COL_PINS[col]) == LOW) {

        delay(20);

        if (digitalRead(COL_PINS[col]) == LOW) {
          return KEYMAP[row][col];
        }
      }
    }
  }

  return '\0';
}
```

9. Bucle principal

```
void loop() {

  char tecla = leerTecla();

  procesarTecla(tecla);

  unsigned long ahora = millis();

  if (ahora - ultimoCicloSensores >= SENSOR_INTERVAL) {
```

```
    ultimoCicloSensores = ahora;

    actualizarSensores();
  }

  delay(5);
}
```

10. Próximo paso recomendado

El siguiente objetivo es terminar de validar el teclado matricial y su integración con el menú. Primero se debe confirmar que el bloque de inicialización de filas/columnas está activo. Después se probará la tecla B y, si no responde correctamente, se hará un programa de diagnóstico para identificar exactamente qué combinación fila/columna produce cada tecla.

Nota de seguridad

En este documento no se incluye la contraseña real del Wi-Fi. En Arduino IDE debe colocarse la clave correspondiente en la variable PASSWORD.